

第7章 图像分割



全红艳

计算机科学与技术学院

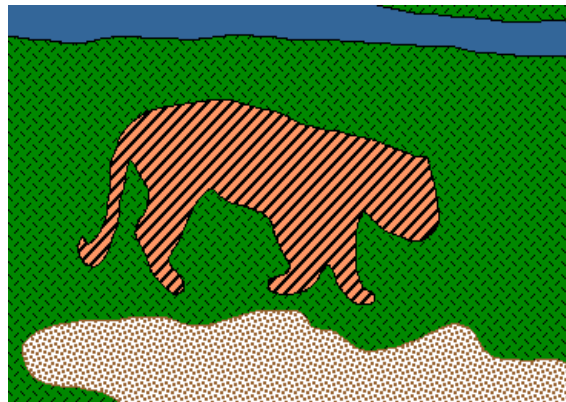
本次课程内容

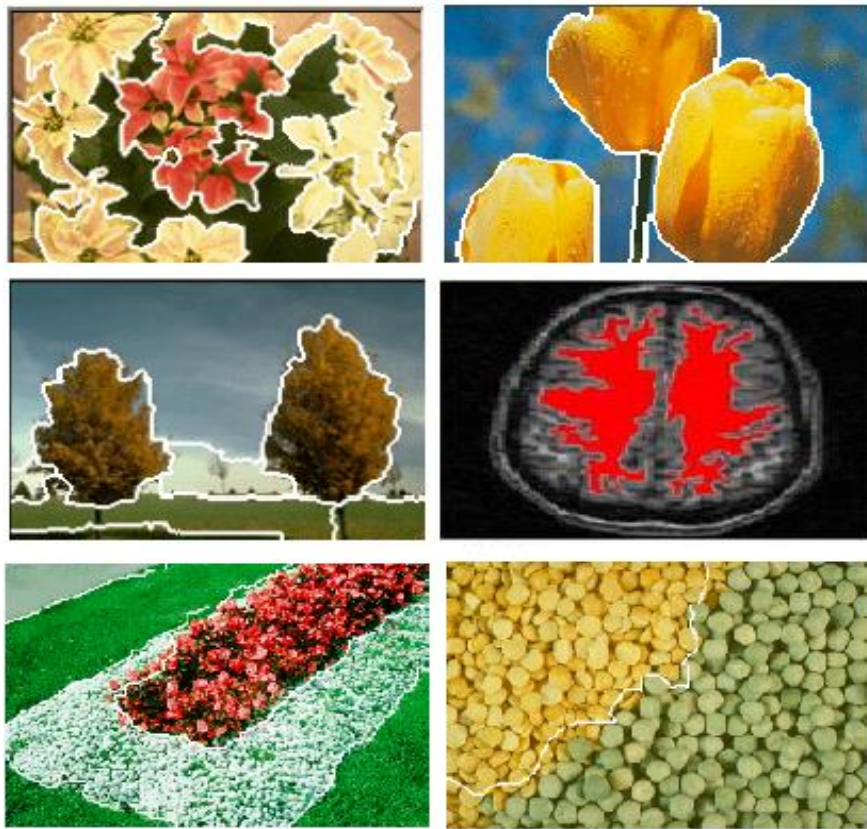
1. 什么是图像分割
2. 图像分割的目标及本质
3. 图像分割算法的种类
4. 区域分割常用算法
5. 图像分割的难点



1. 什么是图像分割

- ◆ 图像分割：就是把图像分成若干个特定的、具有独特性质的区域。
- ◆ 例如：





图像分割的实例

图像分割的描述

- ◆ 目标：将图像划分为若干个子区域 $R_1, R_2, \dots, R_n$ ，这些子区域满足5个条件：

1) 完备性： $\bigcup_{i=1}^n R_i = R$

2) 连通性：每个 R_i 都是一个连通区域

3) 独立性：对于任意 $i \neq j$ ， $R_i \cap R_j = \Phi$

4) 单一性：每个区域内的灰度级相等，

$$P(R_i) = \text{TRUE}, i = 1, 2, \dots, n$$

5) 互斥性：任两个区域的灰度级不等，

$$P(R_i \cup R_j) = \text{FALSE}, i \neq j$$

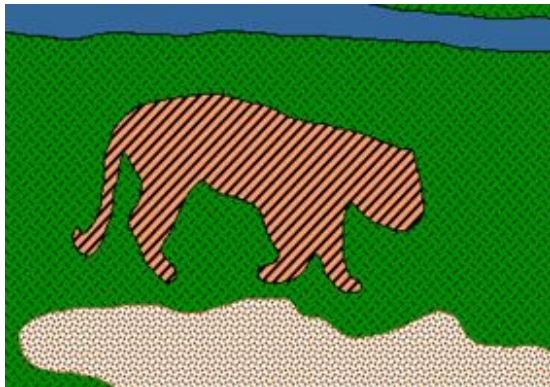


2. 图像分割的目标及本质

◆ 分割目标

- 把人们感兴趣的部分从图像中提取出来
- 有选择地对感兴趣对象定位

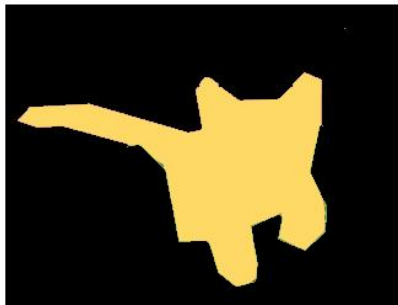
◆ 分割的本质：对图像上的每一个像素点分类



图像分4个区域，任意一个像素属于哪个类别？

分割和语义分割

- ◆ 分割：传统的分割(灰度、颜色)
- ◆ 深度学习：语义分割



3. 图像分割算法的种类

- ◆ 基于阈值分割方法
- ◆ 基于边缘分割方法
- ◆ 基于区域分割方法
- ◆ 基于能量分割方法
- ◆ 基于图论分割算法
- ◆ 基于学习分割方法



4. 区域分割常用算法

- ◆ 阈值分割法
- ◆ 区域生长方法
- ◆ 分裂合并方法
- ◆ 能量方法：蛇模型、水平集方法
- ◆ 聚类分割方法
- ◆ 图割



(1) 阈值分割法

- ◆ 阈值分割法

- 灰度化
- 二值化

- ◆ 阈值确定方法：

- 简单阈值：直方图
- 全局阈值迭代方法
- OTSU方法（最大类间方差法）



阈值分割路线

- ◆ 基本途径：找到灰度值（阈值）或灰度值区间作为前景像素与背景像素的分界
- ◆ 阈值分割的种类：
 - 单阈值分割
 - 多阈值分割



全局阈值确定方法

- ◆ 固定阈值方法（根据直方图直观给出）
- ◆ 全局阈值迭代方法
- ◆ OTSU方法（最大类间方差法）



(a) 全局阈值迭代方法的步骤

选择一个初始化的阈值 T_p (通常取所有像素灰度值的平均值)

2. 使用阈值 T 将图像所有像素分为两集合 G_1 和 G_2 : G_1 包含灰度满足大于 T_p 的像素, G_2 包含灰度满足小于 T_p 的像素。

3. 计算 G_1 中所有像素灰度的均值 μ_1 , 计算 G_2 中所有像素的均值 μ_2

4. 进一步计算当前阈值 $T_N = \frac{\mu_1 + \mu_2}{2}$

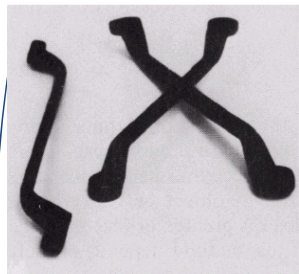
5. 重新步骤2-4, 直到在前后两次计算阈值的差值 $|T_p - T_N|$ 小于一个预先指定的阈值误差 E_T 为止。



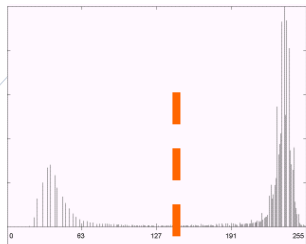
全局阈值迭代分割算法的结果



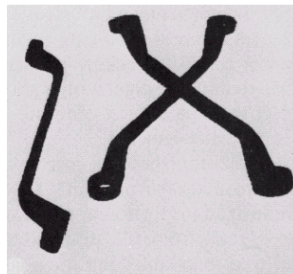
全局阈值迭代法实例



源图像



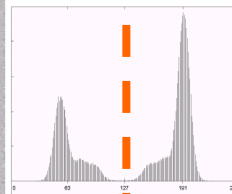
直方图



全局阈值迭代法分割的结果



源图像



直方图



全局阈值迭代法分割的结果

(b) OTSU方法（最大类间方差法）

- ◆ OTSU 算法是自适应地计算单阈值，用来转换灰度图像为二值图像，
- ◆ 该方法简单有效
- ◆ 主要思想：对输入的灰度图像的直方图进行分析，将直方图分成两个部分，使得两部分之间的距离最大。
- ◆ 直方图上灰度的划分点就是求得的阈值。



OTSU算法思想

- ◆ 假设图像灰度值范围为 $[0, 255]$, 并且使用灰度值 T_i 作为阈值进行分割时, 所得到两个区域 G_1 和 G_2 , 它们之间的方差为 D_i 。那么所求的阈值 T 为:

$$T = \left\{ T \mid j \in \max_{j=0}^{j \leq 255} D_j \right\}$$



实现步骤为:

1. 对已知图像计算归一化的直方图, 将其直方图成分记为 p_i , $i = 0, 1, 2, \dots, L-1$;
2. 利用 p_i 计算累计直方图 $P_i(k)$, $i = 1, 2$, $k = 0, 1, 2, \dots, L-1$;
3. 计算累计的均值 m_i , $i = 1, 2$;
4. 计算整体的均值为: $m_G = P_1 m_1 + P_2 m_2$;
5. 计算类间方差 σ_B^2 ;
6. 求取 $k^* = \arg \max_k \sigma_B^2$;
7. 利用阈值 k^* 对图像进行分割。



$$p_i = \frac{n_i}{MN} \quad \sum_{i=0}^{L-1} p_i = 1$$

$$P_1(k) = \sum_{i=0}^k p_i \quad P_2(k) = \sum_{i=k+1}^{L-1} p_i = 1 - P_1(k)$$

C_1 类内像素的平均强度为:

$$m_1(k) = \sum_{i=0}^k iP(i/C_1) = \frac{1}{P_1(k)} \sum_{i=0}^k ip_i$$

C_2 类内像素的平均强度为:

$$m_2(k) = \sum_{i=k+1}^{L-1} iP(i/C_2) = \frac{1}{P_2(k)} \sum_{i=k+1}^{L-1} ip_i$$

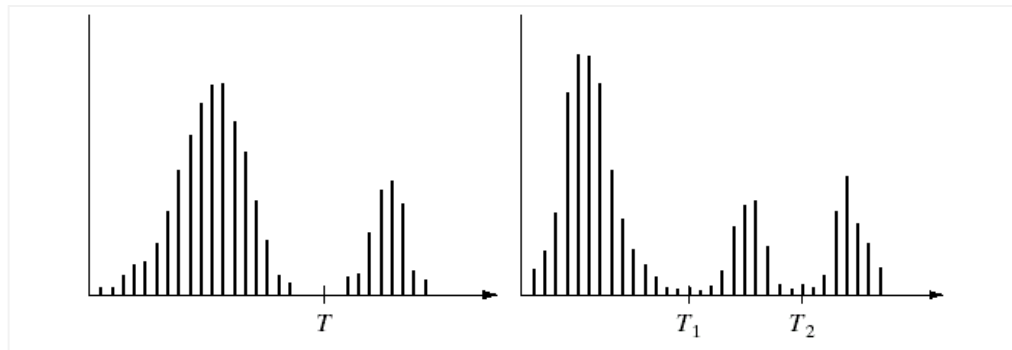
整体的均值为: $m_G = P_1 m_1 + P_2 m_2$

类间的方差定义为: $\sigma_B^2 = P_1(m_1 - m_G)^2 + P_2(m_2 - m_G)^2$



全局阈值法中存在的问题

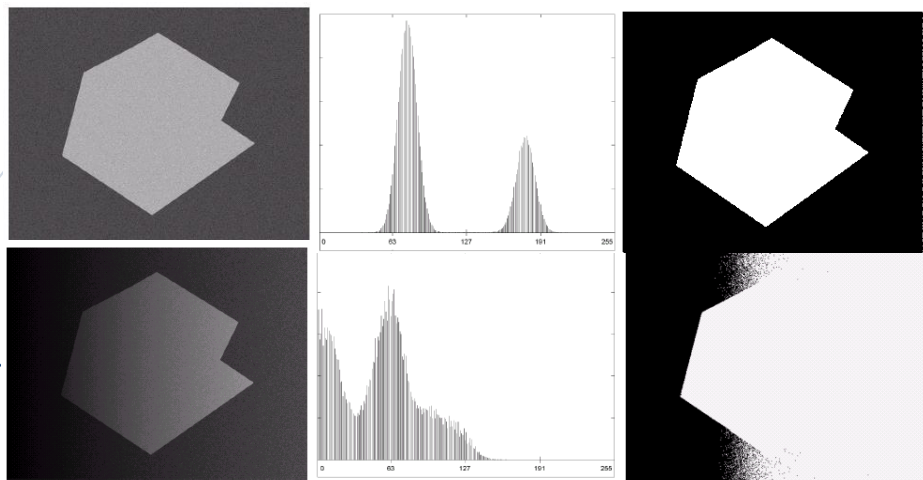
- ◆ 全局阈值法由于采用单一阈值，它仅适合于双峰值的情况
- ◆ 其它类型的直方图则需要多峰值处理



如果用单一阈值会如何？



单一阈值不适合对非均匀光照的处理



- ◆ 不均匀照射, 物体背景对比明显, 但使用一门限不合适。
- ◆ 解决方法
 - 灰度级校正
 - 图像分成小块, 选择局部门限

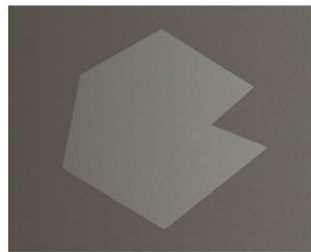
多个阈值的处理方法

- ◆ 对每个像素确定以它为中心的一个邻域窗口，
- ◆ 计算窗口内像素的最大和最小值，
- ◆ 然后取它们的均值作为阈值；

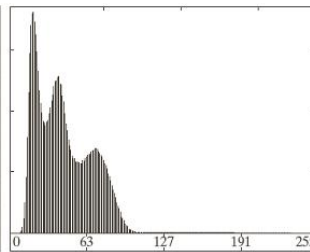
计算窗口内像素的阈值：

1、迭代法

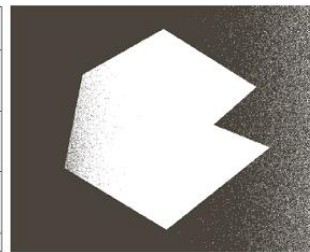
2、OTSU 算法



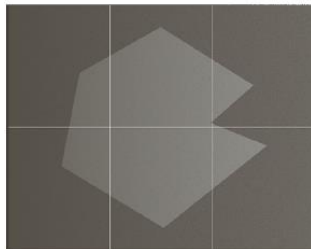
(a) 源图像



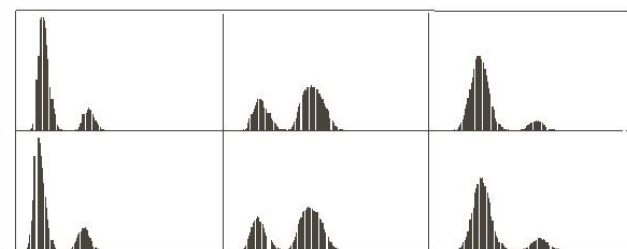
(b) 源图像直方图



(c) 利用全局阈值分割的结果



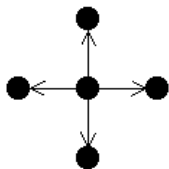
(d) 分割的六个矩形区域



(e) 六个矩形区域的直方图

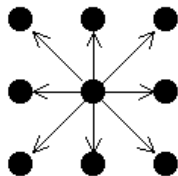
(2) 区域生长方法

◆ 4连通:



➤ 从某一个像素出发，可以通过上、下、左、右邻域到达其它像素

◆ 8连通

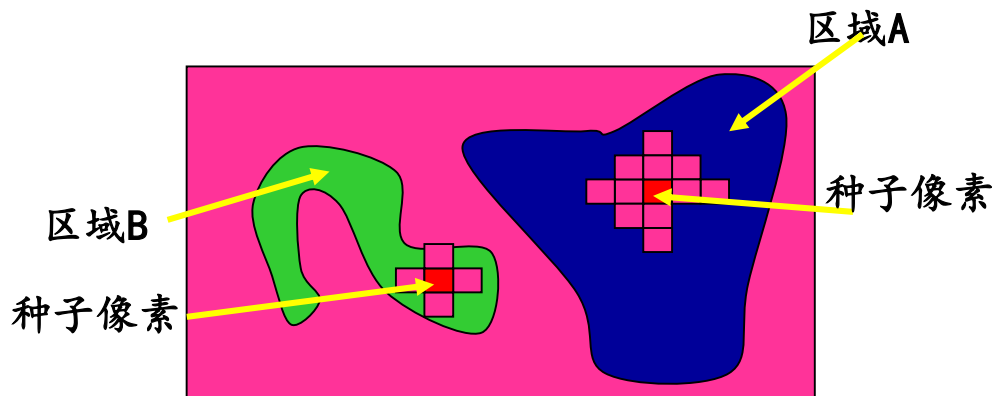


➤ 从某一个像素出发，可以通过上、下、左、右、左上、左下、右上、右下的邻域到达其它像素



(2) 区域生长分割算法

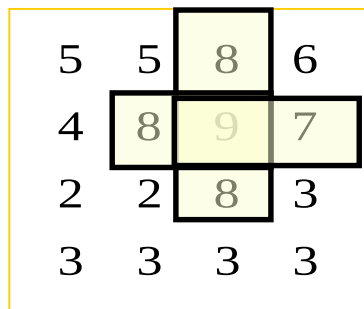
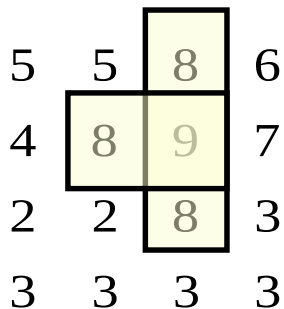
- 1) 区域生长算法：从一个“种子”开始，将图像中的相邻像素的属性（灰度或颜色）与种子进行比较，如果属性相似，就可以将邻像素附加到生长区域中。
- 2) 重复上述的生长过程，直到不再有满足条件的新结点为止



区域生长算法举例

- ◆ 第一步 选择种子点
- | | | | |
|---|---|---|---|
| 5 | 5 | 8 | 6 |
| 4 | 8 | 9 | 7 |
| 2 | 2 | 8 | 3 |
| 3 | 3 | 3 | 3 |

- ◆ 从满足检测准则的点开始(或者已知点)在各个方向上生长出区域。
- ◆ 例如：每一步所接受的邻近点的灰度级与先前物体的平均灰度级相差小于2。



邻域类型影响分割结果

- ◆ 按照4邻域和8邻域进行生长，结果有所不同。
- ◆ 按照8邻域进行生长能够得到较精确的结果

10	10	10	10	10	10	10
10	10	10	69	70	10	10
59	10	60	64	59	56	60
10	59	10	<u>60</u>	70	10	62
10	60	59	65	67	10	65
10	10	10	10	10	10	10
10	10	10	10	10	10	10

10	10	10	10	10	10	10
10	10	10	69	70	10	10
59	10	60	64	59	56	60
10	59	10	<u>60</u>	70	10	62
10	60	59	65	67	10	65
10	10	10	10	10	10	10
10	10	10	10	10	10	10

10	10	10	10	10	10	10
10	10	10	69	70	10	10
59	10	60	64	59	56	60
10	59	10	<u>60</u>	70	10	62
10	60	59	65	67	10	65
10	10	10	10	10	10	10
10	10	10	10	10	10	10

4邻域及8邻域生长的结果比较



(3) 分裂合并分割算法

◆ 区域分裂合并算法的基本思想是：

- 1) 先确定一个分裂合并的准则，即区域特征一致性的测度
- 2) 当图像中某个区域的特征不一致时就将该区域分裂成4 个相等的子区域 ；
- 3) 当相邻的子区域满足一致性特征时，则将它们合成一个大区域；
- 4) 重复进行步骤（1）和（2）直至所有区域不再满足分裂合并的条件为止



- ◆ 考察各块图像的灰度是否均匀的方法
- ◆ 方法：均方差
- ◆ 计算方法：

$$E_c = \sum_{(x,y) \in A} \sum [f(x,y) - m]^2$$



分裂合并算法实例

(1) 分裂时一致性的规则为：

- 如果某个子区域的灰度均方差大于1.5，则将其分裂成4个子区域，否则不分裂。

(2) 合并时的相似性规则为：

- 如果相邻两个子区域灰度均方差小于等于2.5，则合并为一个区域。



分裂合并算法实例

5	5	8	6
4	8	9	7
2	2	8	3
3	3	2	2

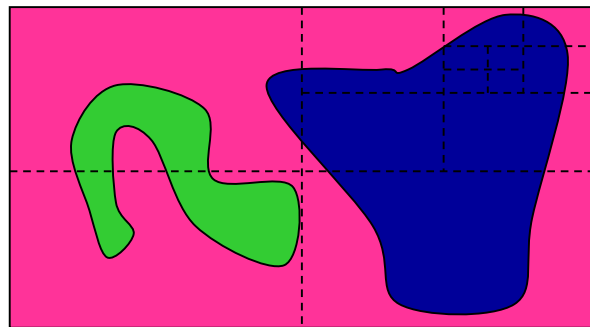
$$\sigma_R = 2.65$$

$$\mu_{R1} = 5.5, \sigma_{R1} = 1.73;$$

$$\mu_{R3} = 2.5, \sigma_{R3} = 0.25;$$

$$\mu_{R2} = 7.5, \sigma_{R2} = 1.29$$

$$\mu_{R4} = 3.75, \sigma_{R4} = 2.87$$



分割结果

R_1	R_2
R_3	R_4

(a) 第一次分裂结果

R_{11}	R_{12}	R_2	
R_{13}	R_{14}		
R_3		R_{41}	R_{42}
		R_{43}	R_{44}

(b) 第二次分裂结果

5	5	8	6
4	8	9	7
2	2	8	3
3	3	3	3

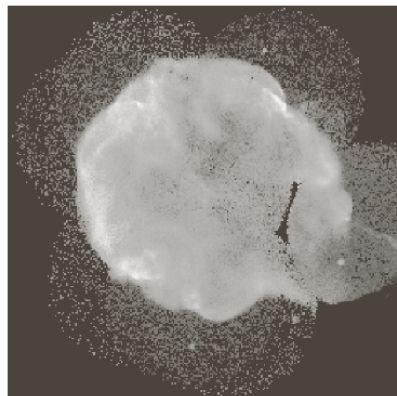
(c) 第一次合并结果

5	5	8	6
4	8	9	7
2	2	8	3
3	3	3	3

(d) 最后的分割结果



源图像



最小块尺度

32×32

最小块尺度

16×16

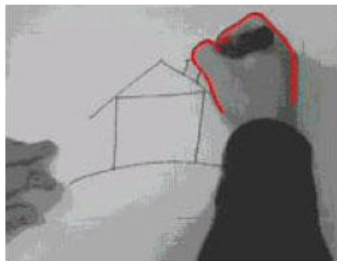


最小块尺度

8×8

(4) 能量方法：蛇模型

◆ 应用背景



Human-computer interaction



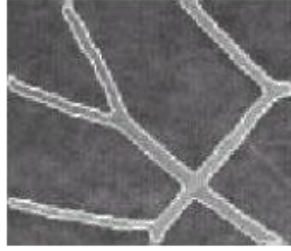
Surveillance/speed control



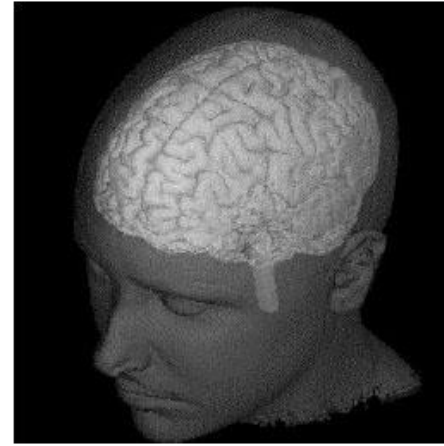
Lip-reading



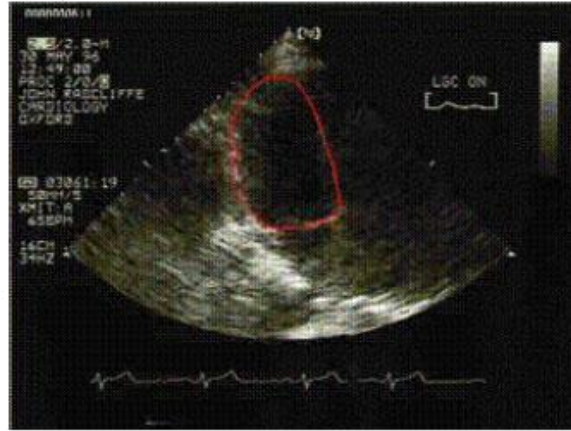
Face recognition



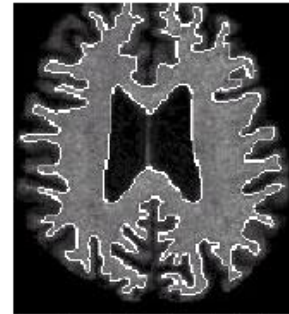
Blood vessels



3D brain



Heart in ultrasound

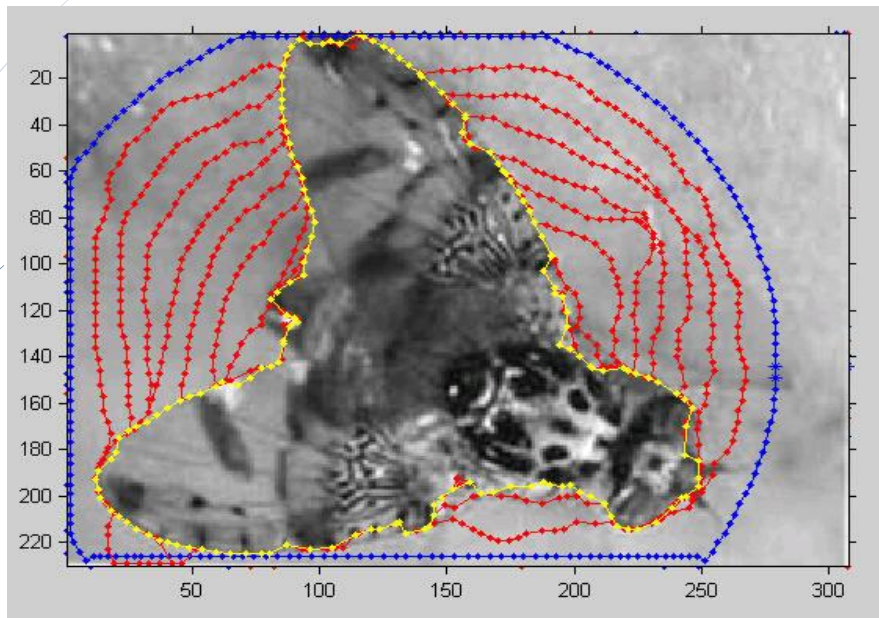


brain in MR



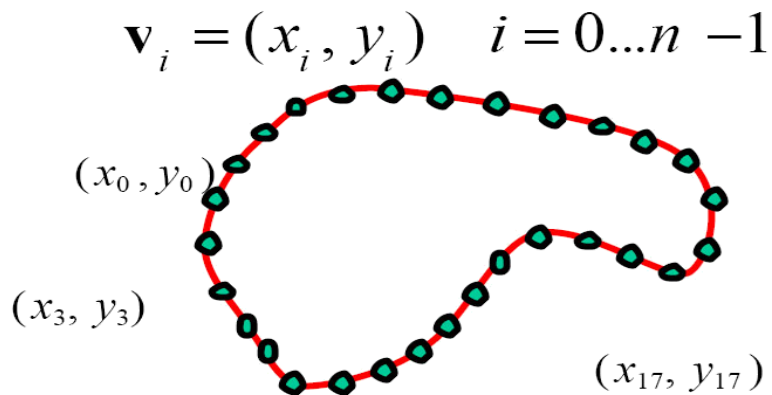
适合用活动轮廓线模型处理的实例

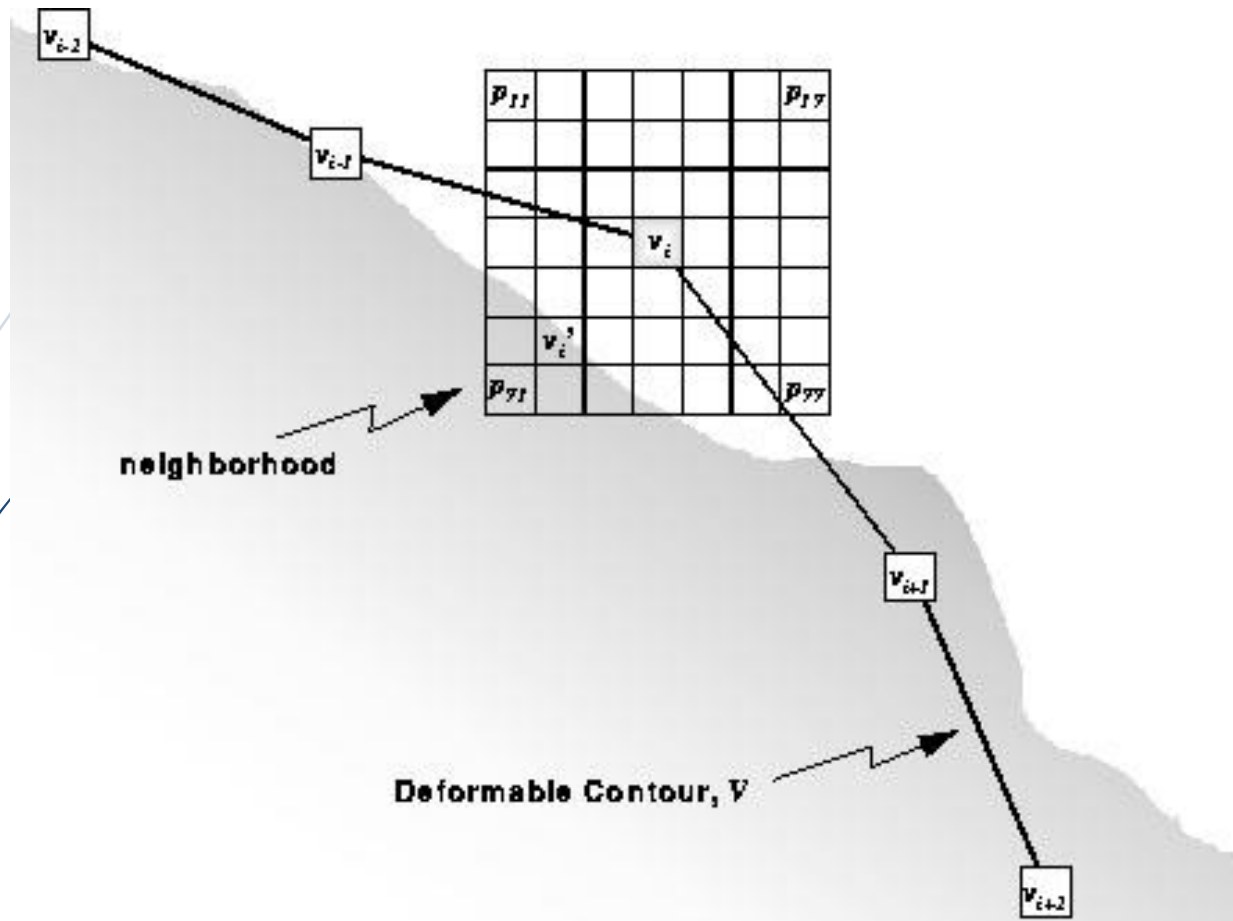
利用蛇模型演化得到边缘



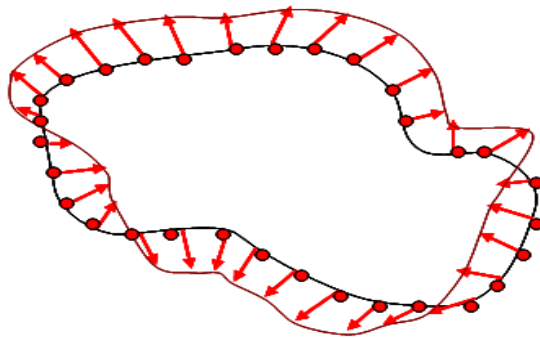
蛇模型算法概述

- ◆ 活动轮廓线上的控制点 $V = \{v_0, v_1, \dots, v_{n-1}\}$





- ◆ 主动轮廓线的演化过程就是顶点序列的迭代过程，在每次迭代时，得到顶点序列新的位置，并且计算得到新的参数



Snake算法的能量公式

- ◆ 在该曲线上能量函数定义为：

$$E_{snake} = E_{external} + E_{internal}$$

- ◆ 能量函数由两部分组成：

- 外能量 $E_{external}$ ：用以靠近目标物体边缘。
- 内能量 $E_{internal}$ ：用以控制轮廓的平滑性和连续性。



外部能量计算方法

- ◆ 外部能量可以从图像的强度(灰度)及边缘特征获得,它是用来吸引曲线到目标边缘的动力

$$E_{external} = -|\nabla I(x, y)|^2$$

- ◆ 其中 $\nabla I(x, y)$ 表示图像在该控制点处的梯度



外部能量计算方法

外部能量 $E_{external}$ 可以从图像的强度(灰度)及边缘特征获得, 是用来吸引曲线到目标边缘的动力。对于图像 $I(x, y)$, $E_{external}$ 可以从 $I(x, y)$ 得到连续函数, 即:

$$E_{external} = -|\nabla I(x, y)|^2$$

其中 $\nabla I(x, y)$ 表示图像在该控制点处的梯度。如果图像有噪声, 可以通过与滤波器 $G_\sigma(x, y)$ 卷积去噪后再求梯度, 即:

$$E_{external} = -|G_\sigma(x, y) * \nabla I(x, y)|^2$$



内部能量计算方法

- ◆ 内部能量是建立在曲线的本身属性上的。
- ◆ 其值为曲线的弹性势能和弯曲势能之和


$$E_{elastic} = \int_0^1 \frac{1}{2} \alpha(s) |v'(s)|^2 ds$$


$$E_{blending} = \int_0^1 \frac{1}{2} \beta(s) |v''(s)|^2 ds$$



内部能量计算方法

内部能量是建立在曲线的本身属性上的，其值为曲线的弹性势能和弯曲势能之和。曲线若被看成为一块有弹性的橡皮，当有外力使其伸展时，就产生弹性势能使其收缩。曲线的弹性势能定义如下：

$$E_{elastic} = \int_0^1 \frac{1}{2} \alpha(s) |v'(s)|^2 ds$$

其中 $v'(s)$ 是控制点 v 对 s 的一阶导数， $\alpha(s)$ 为弹性系数，用于控制曲线的弹性。

曲线若被看作为一块薄的钢板，其弯曲势能定义为曲线上各个点的曲率之和：

$$E_{blending} = \int_0^1 \frac{1}{2} \beta(s) |v''(s)|^2 ds$$

其中 $|v''(s)|$ 是控制点 v 对 s 的二阶导数， $\beta(s)$ 用于控制曲线的刚性；当曲线为一个圆时，曲线的弯曲势能为最小。
由此，内部能量可写为：

$$E_{internal} = E_{elastic} + E_{blending} = \int_0^1 \frac{1}{2} \alpha(s) |v'(s)|^2 ds + \int_0^1 \frac{1}{2} \beta(s) |v''(s)|^2 ds$$



实例

$$p_{1,1} = I(3,1), p_{1,2} = I(4,1), p_{1,3} = I(5,1), p_{2,1} = I(3,2), p_{2,2} = I(4,2)$$

$$p_{2,3} = I(5,2), p_{3,1} = I(3,3), p_{3,2} = I(4,3), p_{3,3} = I(5,3)$$

$$v_{i-1} = I(0,0), v_i = I(4,2), v_{i+1} = I(3,5)$$

三个控制点

	0	1	2	3	4	5
0	100 v_{i-1}	110	100	130	130	130
1	100	90	100	110	120	120
2	65	90	105	90	105 v_i	110
3	60	50	80	90	110	120
4	50	55	50	70	100	120
5	60	70	50	90 v_{i+1}	100	130



弹力势能 $E_{elastic}$

$$E_{elastic} = -\nabla E_{ext} = n_i (v_i - p_{j,k})$$

切向量 t_i 为:

$$t_i = \frac{v_i - v_{i-1}}{\|v_i - v_{i-1}\|} + \frac{v_{i+1} - v_i}{\|v_{i+1} - v_i\|} = \frac{(4,2)}{\sqrt{20}} + \frac{(-1,3)}{\sqrt{10}} = (0.6, 1.4)$$

可得法向量 n_i 为:

$$n_i = (-1.4, 0.6)$$

$$E_{elastic}(p_{1,1}) = (-1.4, 0.6) \times (1, 1) = -0.8$$

类似地, ↻

$$E_{elastic}(p_{1,2}) = 0.6, E_{elastic}(p_{1,3}) = 2.0, E_{elastic}(p_{2,1}) = -1.4, E_{elastic}(p_{2,2}) = 0;$$



弯曲势能:

$$E_{blending} = \beta(s)v'''(s) = \left\| p_{j,k} - \frac{1}{2}(v_{i-1} + v_{i+1}) \right\| = \sqrt{(x_{p_{j,k}} - 1.5)^2 + (y_{p_{j,k}} - 2.5)^2}$$

$$E_{blending}(p_{1,1}) = \sqrt{(3 - 1.5)^2 + (1 - 2.5)^2} \approx 2.1$$

类似地, ↙

$$E_{blending}(p_{1,2}) \approx 2.9, \quad E_{blending}(p_{1,3}) \approx 3.8, \quad E_{blending}(p_{2,1}) \approx 1.6,$$

$$E_{blending}(p_{2,2}) \approx 2.5, \quad E_{blending}(p_{2,3}) \approx 3.5, \quad E_{blending}(p_{3,1}) \approx 1.6,$$

$$E_{blending}(p_{3,2}) \approx 2.5, \quad E_{blending}(p_{3,3}) \approx 3.5$$



$$E_{\text{external}} = \alpha(s) v''(s) = \|\nabla I(p_{j,k})\| = \sqrt{(I(p_{j,k}) - I(p_{j-1,k}))^2 + (I(p_{j,k}) - I(p_{j,k-1}))^2}$$

外部能量 E_{external} :

$$E_{\text{external}}(p_{j,k}) = \alpha(s) v''(s) = \|\nabla I(p_{j,k})\| = \sqrt{(I(p_{j,k}) - I(p_{j-1,k}))^2 + (I(p_{j,k}) - I(p_{j,k-1}))^2}$$

$$E_{\text{external}}(p_{1,1}) = \sqrt{(110 - 100)^2 + (110 - 130)^2} \approx 22.4$$

类似地,

$$E_{\text{external}}(p_{1,2}) \approx 14.1, \quad E_{\text{external}}(p_{1,3}) = 10, \quad E_{\text{external}}(p_{2,1}) = 25, \quad E_{\text{external}}(p_{2,2}) \approx 21.2 ;$$

$$E_{\text{external}}(p_{2,3}) \approx 11.1, \quad E_{\text{external}}(p_{3,1}) \approx 10, \quad E_{\text{external}}(p_{3,2}) \approx 20.6, \quad E_{\text{external}}(p_{3,3}) \approx 14.1$$



加权计算总能量

$$E(p_{i,j}) = \alpha E_{\text{external}}(p_{i,j}) + \beta E_{\text{blending}}(p_{i,j}) + \gamma E_{\text{elastic}}(p_{i,j})$$

$$E(p_{i,j}) = -0.1 \times E_{\text{external}}(p_{i,j}) + 1 \times E_{\text{blending}}(p_{i,j}) - 1 \times E_{\text{elastic}}(p_{i,j})$$

可得:

$$E(p_{1,1}) = -0.1 \times 22.4 + 1 \times 2.1 - 1 \times 2.0 = -2.14$$

类似地可以计算,

$$E(p_{1,2}) = -0.1 \times 14.1 + 1 \times 2.9 - 0.6 = 0.89, \quad E(p_{1,3}) = -0.1 \times 10 + 1 \times 3.8 - 2 = 0.8,$$

$$E(p_{2,1}) = -0.1 \times 25 + 1 \times 1.6 - 1.4 = -2.3, \quad E(p_{2,2}) = -0.1 \times 2.24 + 2.5 - 0 = 0.26,$$

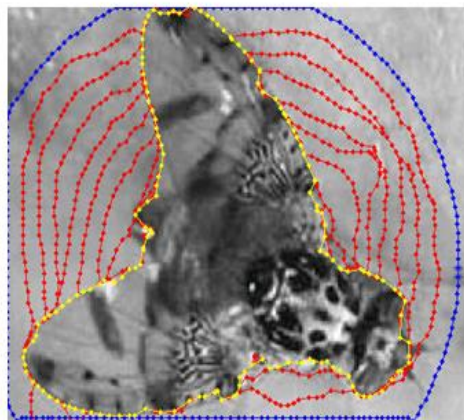
$$E(p_{2,3}) = -0.1 \times 11.1 + 3.5 + 1.4 = 3.79, \quad E(p_{3,1}) = -0.1 \times 10 + 1.6 - 2 = -1.4,$$

$$E(p_{3,2}) = -0.1 \times 20.6 + 2.5 + 0.6 = 1.04, \quad E(p_{3,3}) = -0.1 \times 14.1 + 3.5 + 2.0 = 4.09$$

可见最小值为 $E(p_{2,1})$, 那么当前控制点演化到 $p_{2,1}$ 点处, 同理可以求得当前演化边缘

上的所有控制点演化到的位置, 这样所有控制点得到了新的演化位置, 演化曲线的本次演化就结束, 进入下一演化过程。这样经过若干次演化后, 直到演化曲线停止到边缘为止。

下图 是演化得到结果。



活动轮廓线演化得到边缘结果

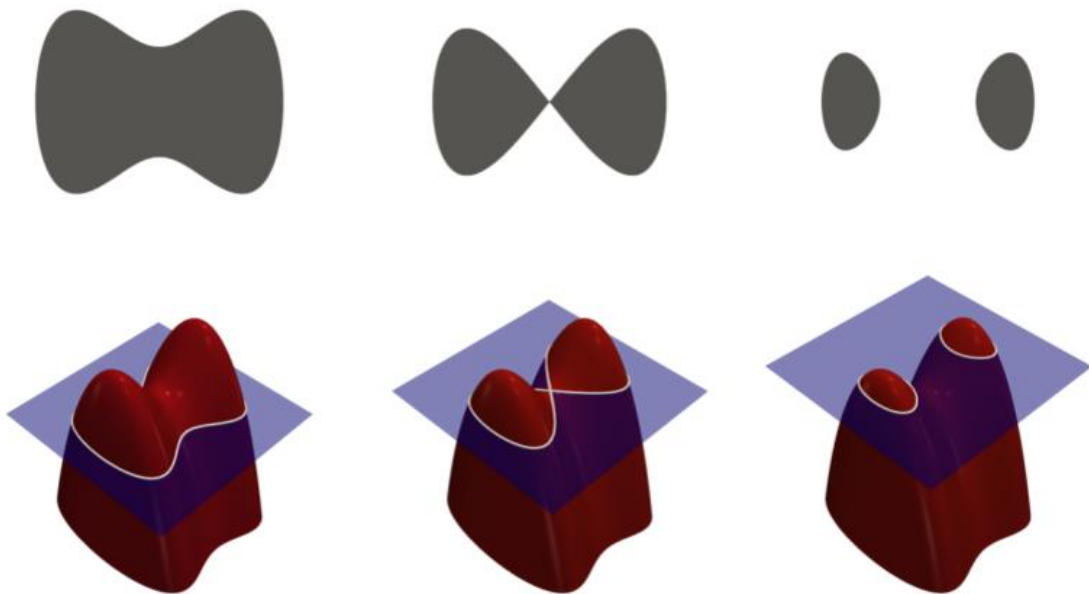
(4) 能量方法：水平集模型

- ◆ 利用水平集方法计算曲线、曲面的演化，而无需知道曲线曲面的参数，所以这种方法又称为几何法。
- ◆ 在图像分割任务中，需要对描述曲线在xoy坐标系下的演化。最初曲线在二维图像平面上呈现： $y=g(x)$
- ◆ 分割时，动态将这个曲线看成是三维曲面下的一条等高线， $z=f(x, y)$ 可以看作是一个xoy平面的隐函数方程。



水平集模型

- ◆ 特别的, $z=f(x, y)$ 设置为0时, 得到零水平集就是图像上的边缘包络

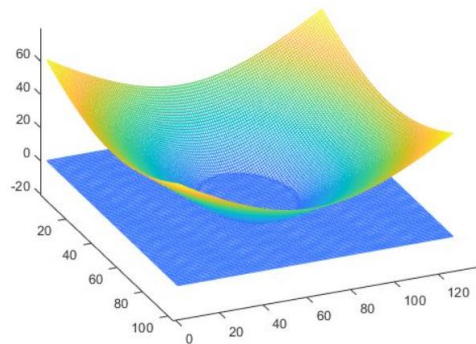


符号距离函数

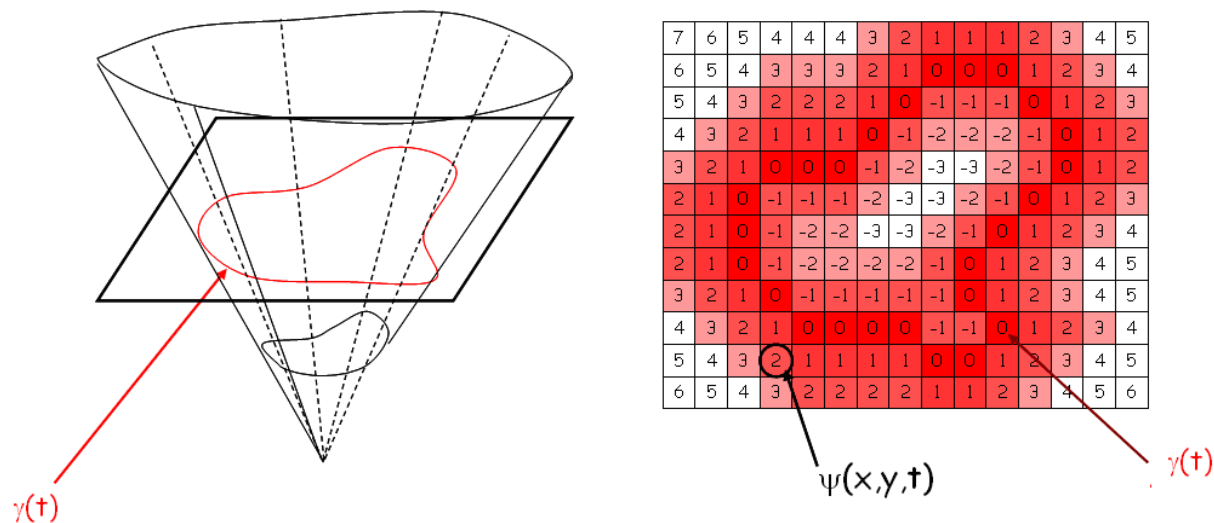
$$\phi(x, t) = \begin{cases} < 0, & x \in \Omega & \text{点在曲线边缘的内侧} \\ = 0, & x \in \partial\Omega & \text{点在曲线边缘上} \\ > 0, & x \notin \Omega & \text{点在曲线边缘的外侧} \end{cases}$$

其中， t 表示时间， x 是水平集设计变量， ϕ 是水平集函数， Ω 是作用域（感兴趣区域）， $\partial\Omega$ 是作用域的边界

- 求解边缘的过程就是水平集能量最小化的过程
- 在水平集演化过程中将符号距离函数作为推动力
- 不断演化
- 当符号距离函数为0时，得到理想边缘

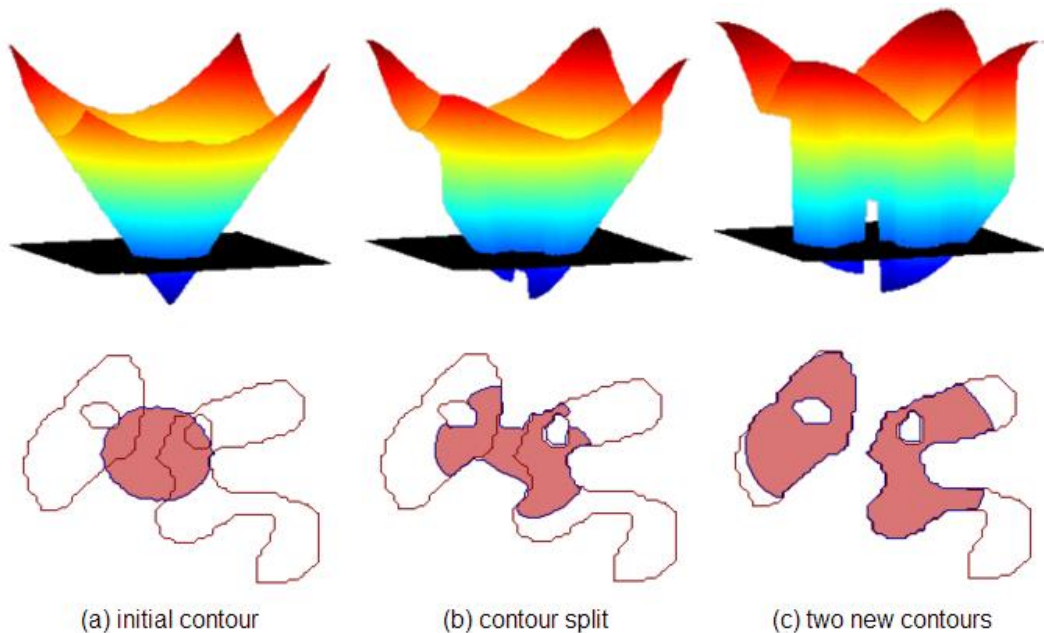


水平集演化过程



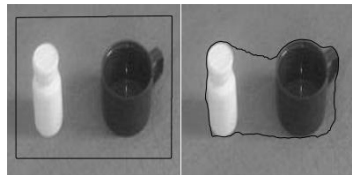
水平集模型

- ◆ 在数学中可以归结为能量泛函问题，求解边缘的过程就是这个水平集能量泛函最小化的过程。



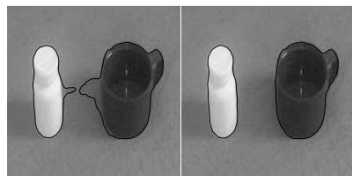
水平集模型

- ◆ 典型工作: Level Set Evolution Without Re-initialization: A New Variational Formulation
- ◆ 由符号距离函数构成的水平集函数偏差作为内部能量项, 水平集向目标图像运动特征作为外部能量项进行驱动, 实现很好的分割效果。



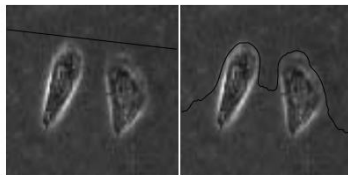
(a) The initial contour.

(b) 200 iterations



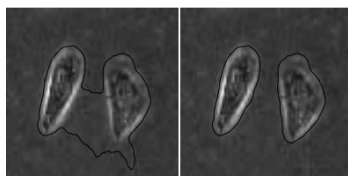
(c) 450 iterations

(d) 650 iterations



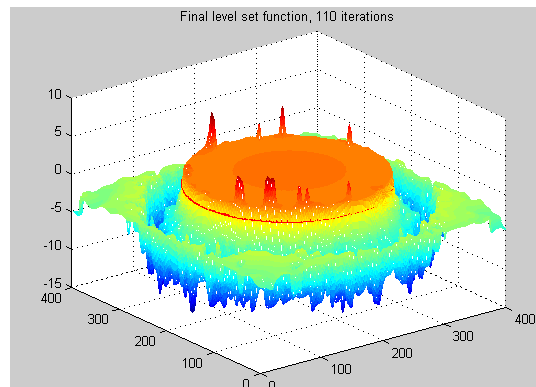
(a) The initial contour.

(b) 200 iterations



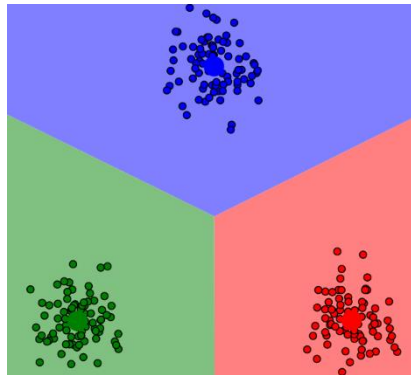
(c) 600 iterations

(d) 800 iterations



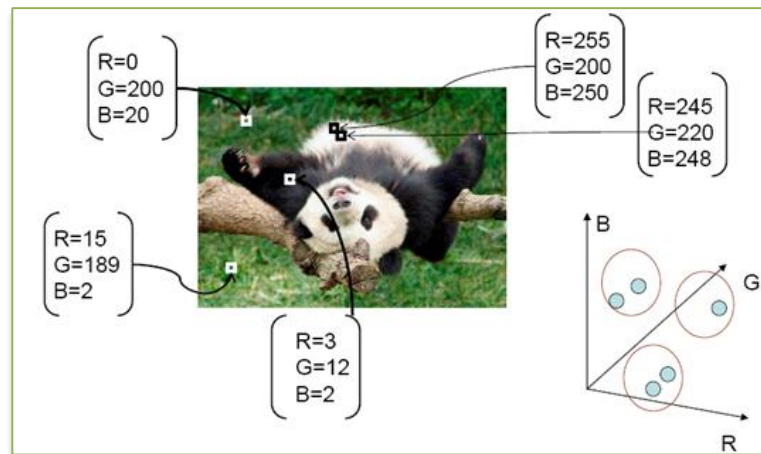
(5) 聚类分割方法

- ◆ 聚类概念：特征无监督学习，没有标签。
- ◆ 将一系列无标签的数据（像素）输入到算法中，然后根据数据之间的相似性，将它们分成几组（簇），这种能够找到这些簇的算法称为聚类算法



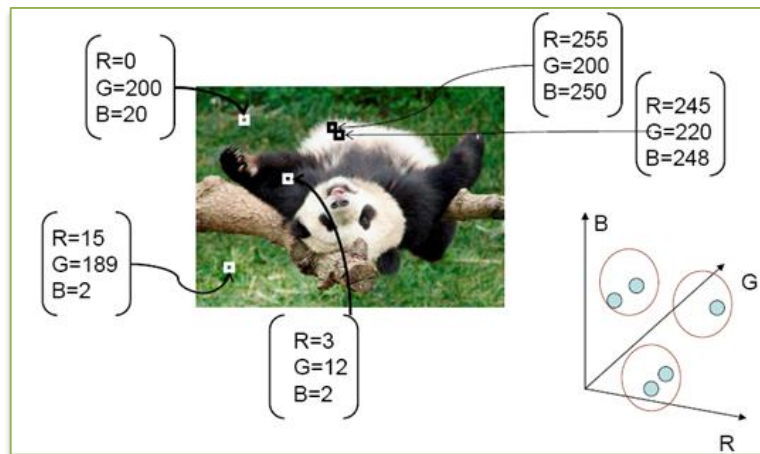
聚类分割方法

- ◆ 特征相似的像素，聚为同一类



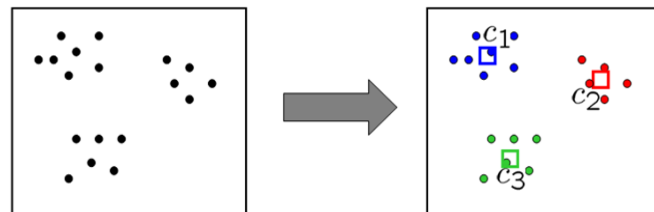
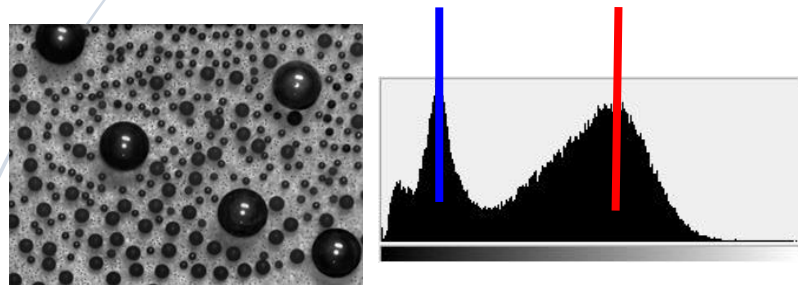
实例：基于颜色的K-means 聚类

- ◆ 特征相似的像素，聚为同一类



实例：基于颜色的K-means 聚类

◆ 直方图确定分割几类



$$\sum_{\text{clusters } i} \sum_{\text{points } p \text{ in cluster } i} \|p - c_i\|^2$$

实例

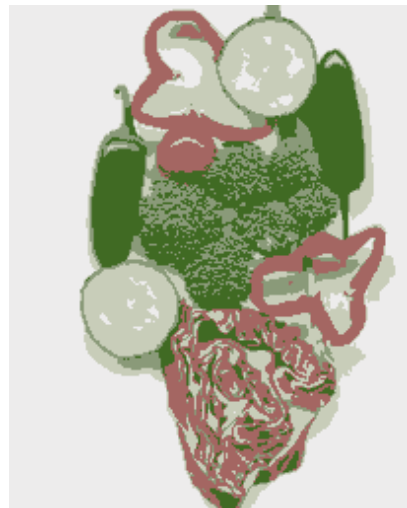
原图像



基于强度聚类

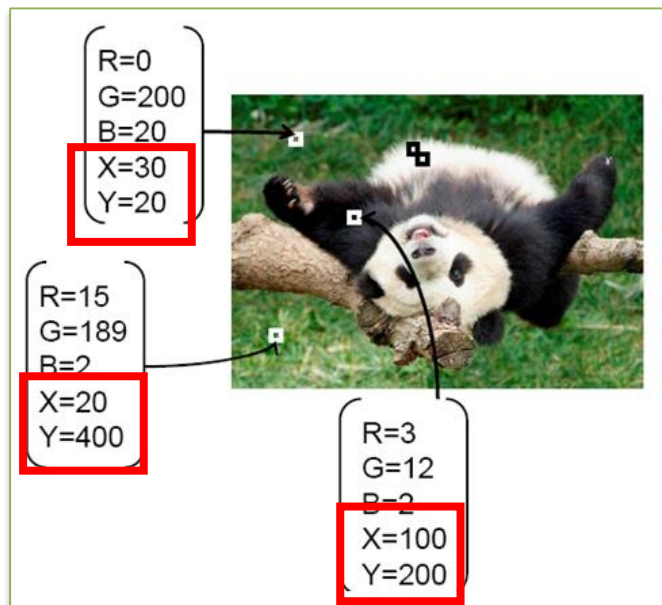
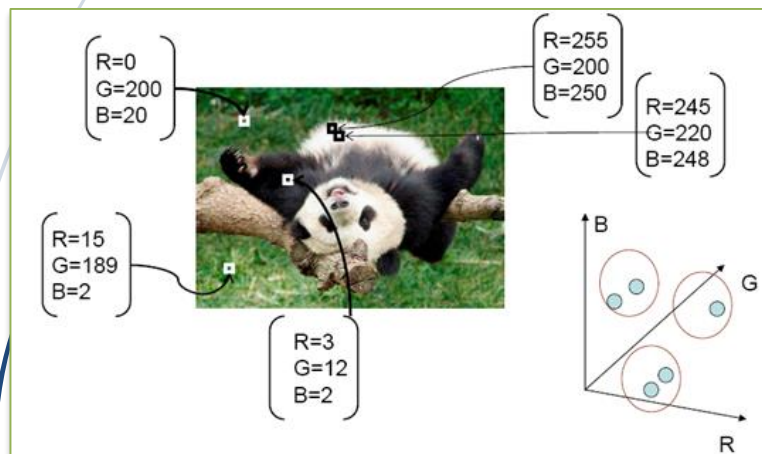


基于颜色聚类



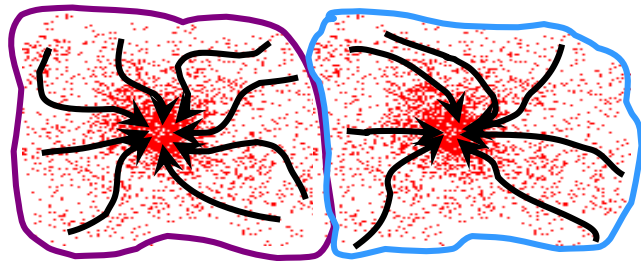
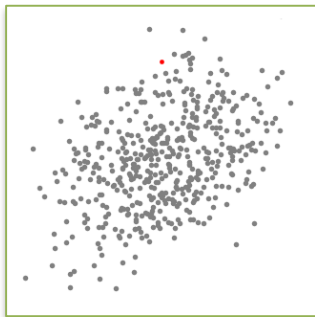
基于K-means聚类问题

- ◆ 问题：没考虑空间特征性，像素位置信息
- ◆ 改进措施，将位置也作为特征的一部分

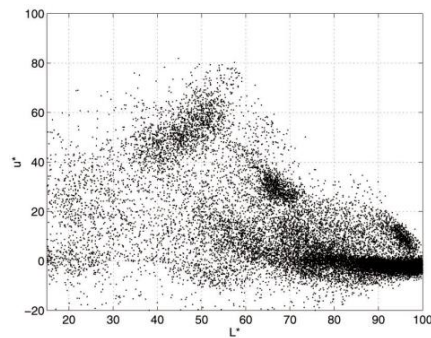


均值漂移聚类

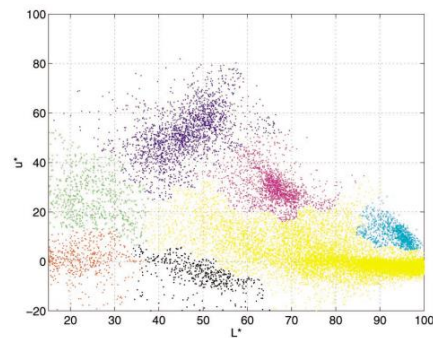
- ◆ 主要思想：从一个以 C 点（随机选择）为中心，以半径 r 为核心的圆形滑动窗口开始。均值漂移是一种爬山算法，它包括在每一步中迭代地向更高密度区域移动，直到收敛。
- ◆ 移动规则：在每次迭代中，滑动窗口通过将中心点移向窗口内点的均值，移向更高密度区域。通过移动，中心点会移向密度更高的区域。



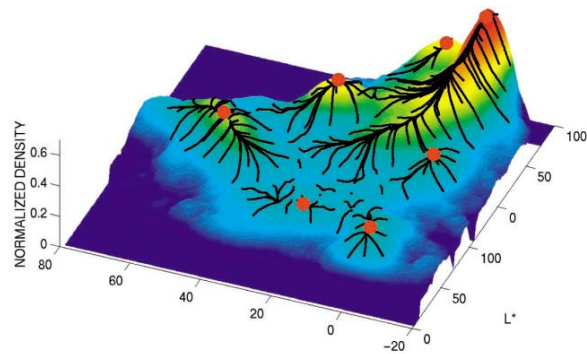
均值漂移聚类结果



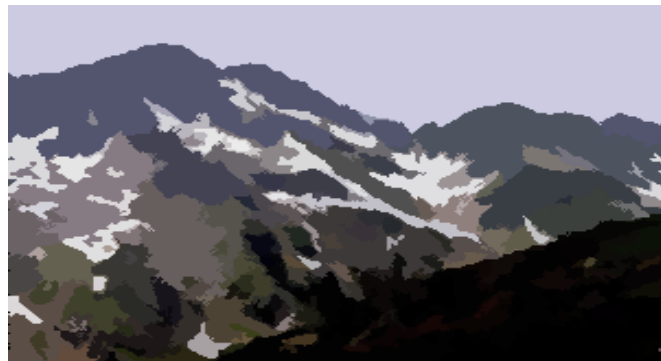
(a)



(b)

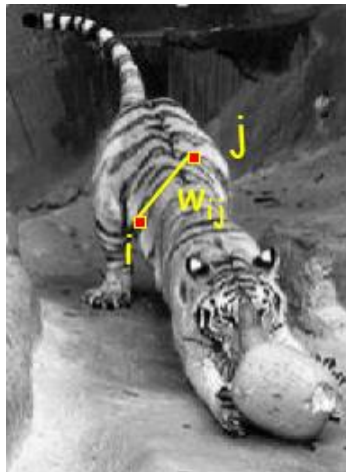
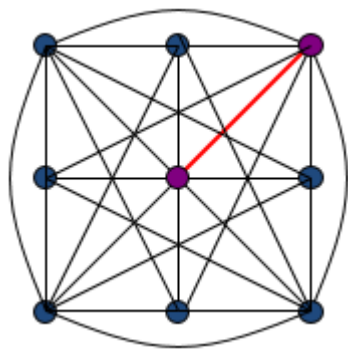


均值漂移聚类结果



(6) 图割 (Graph-based segmentation)

- ◆ GraphCut利用最小割最大流算法进行图像分割，可以将图像分割为前景和背景。
- ◆ 主要思想：建立各个像素点与前景背景相似度的赋权图，并通过求解最小切割区分前景和背景。算法效果图如下：

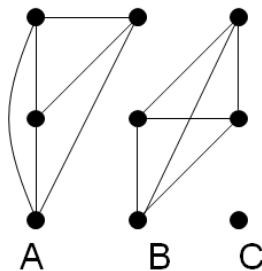


图割 (Graph-based segmentation)

◆ 将亲和关系较弱的图分割开

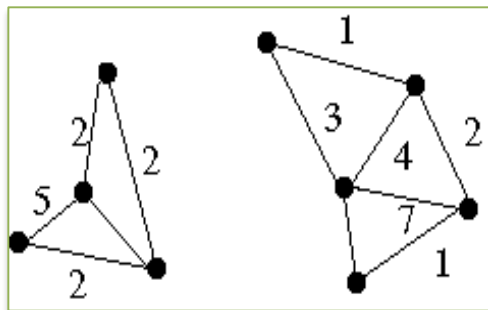
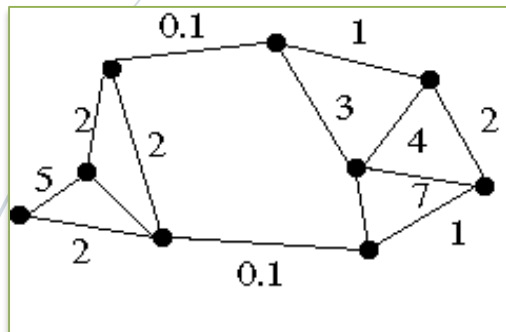
◆ 亲和关系度量方法:

$$\exp\left(-\frac{1}{2\sigma^2} \text{dist}(\mathbf{x}_i, \mathbf{x}_j)^2\right)$$



图割

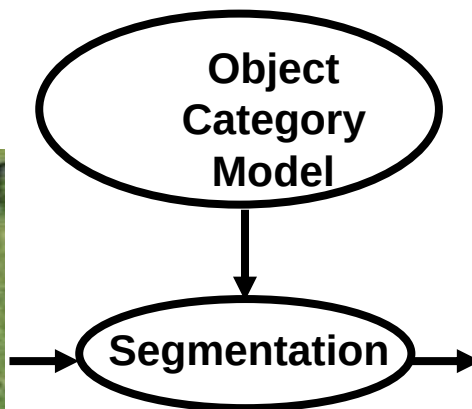
- ◆ 通过求图的最小割来进行分割



实例



Cow Image



Segmented Cow

5. 图像分割问题的难点

类内外观差异很大



自遮挡

失败案例-难以识别物体

